

ThreatOS

Oracle Linux 8 — Deployment Guide

Step-by-step rebuild instructions for a new server

Target OS	Oracle Linux 8.x (RHEL 8 compatible)
Python	3.11.x (pre-installed or <code>dnf install python3.11</code>)
Database	PostgreSQL 15 (existing or fresh install)
Cache	Redis 5.x (<code>dnf install redis</code>)
Web Server	nginx (<code>dnf install nginx</code>)
API Port	8001 (change if already occupied)
UI Port	8080 (change if already occupied)
Install Path	<code>/opt/threatos</code>
Coverage	2,641 Sigma rules 49% ATT&CK; coverage 691 techniques

This guide documents every command run to deploy ThreatOS from scratch on Oracle Linux 8. Follow the steps in order. Each section includes the exact commands, expected output, and notes on issues that were encountered and resolved.

Table of Contents

1. Pre-flight Checks
2. Redis Installation
3. PostgreSQL — Create threatos Database
4. Project Setup — Virtualenv & Dependencies
5. Config Files (.env, alembic.ini, pyproject.toml)
6. Database Migration (Alembic)
7. Source Code Deployment
8. Start the API (uvicorn)
9. Verify All Routes
10. Start the Ingest Worker
11. systemd Service Units
12. Web UI — React Build
13. nginx Configuration
14. Sigma Rules Import
15. Daily Automation (systemd Timer)
16. Smoke Test — End-to-End Pipeline
17. Troubleshooting — Bugs Encountered & Fixed
18. Quick Reference Card

1. Pre-flight Checks

Run these checks first to understand what is already installed and what ports are in use.

Audit commands

```
cat /etc/oracle-release
uname -r
python3.11 --version
which python3.11

# Check ports in use
ss -tlnp | grep -E ':8000|:5432|:6379|:8001|:8080'

# Disk space (need at least 2GB free)
df -h /

# RAM
free -h

# SELinux (disabled = easier; enforcing = extra steps needed)
getenforce

# Firewall
sudo firewall-cmd --state

# Existing services
systemctl status postgresql-15 redis nginx 2>/dev/null | grep -E "Active|●"
```

Note: Port 8000 was already in use by another app (FIM). ThreatOS was deployed on 8001 instead.

2. Redis Installation

Redis is used for the event stream (XREADGROUP) and pubsub (WebSocket relay). Install natively — no Docker needed.

Install and start

```
sudo dnf install -y redis
sudo systemctl enable --now redis
sudo systemctl status redis | head -5
redis-cli ping      # Expected: PONG
```

3. PostgreSQL — Create threatos Database

PostgreSQL 15 was already running. We needed to create a dedicated DB and user without disrupting the existing FIM application.

Find the postgres superuser socket

```
# pg_hba.conf location
find /var/lib/pgsql -name pg_hba.conf

# PostgreSQL runs as numeric UID — no named OS user
ps -p $(cat /var/lib/pgsql/15/data/postmaster.pid | head -1) -o user,cmd
```

Temporarily allow trust login for postgres superuser

Important: Always restore pg_hba.conf immediately after creating the DB. Do not leave trust enabled.

```
# Backup first
cp /var/lib/pgsql/15/data/pg_hba.conf /var/lib/pgsql/15/data/pg_hba.conf.bak

# Add trust line at TOP of file
sed -i '1s/^/local all postgres trust\n/' \
/var/lib/pgsql/15/data/pg_hba.conf

# Reload PostgreSQL (send SIGHUP — pg_ctl cannot run as root)
kill -HUP $(cat /var/lib/pgsql/15/data/postmaster.pid | head -1)

# Test connection via Unix socket (not TCP — TCP still requires password)
psql -U postgres -d postgres -c "SELECT current_user;"
```

Create database and user

```
psql -U postgres -d postgres -c "CREATE USER threatos WITH PASSWORD 'threatos_secret';"
psql -U postgres -d postgres -c "CREATE DATABASE threatos OWNER threatos;"
psql -U postgres -d postgres -c "GRANT ALL PRIVILEGES ON DATABASE threatos TO threatos;"
psql -U postgres -d postgres -c "\l" | grep threatos
```

Restore pg_hba.conf and add threatos auth

```
# Restore original
cp /var/lib/pgsql/15/data/pg_hba.conf.bak /var/lib/pgsql/15/data/pg_hba.conf

# Add threatos TCP auth
cat >> /var/lib/pgsql/15/data/pg_hba.conf << 'EOF'
host threatos threatos 127.0.0.1/32 scram-sha-256
host threatos threatos ::1/128 scram-sha-256
EOF

# Reload
kill -HUP $(cat /var/lib/pgsql/15/data/postmaster.pid | head -1)

# Test TCP connection (will prompt for password: threatos_secret)
psql -h localhost -U threatos -d threatos -c "SELECT current_user;"
```

4. Project Setup — Virtualenv & Dependencies

Create directory and virtualenv

```
sudo mkdir -p /opt/threatos
sudo chown $USER:$USER /opt/threatos
cd /opt/threatos
python3.11 -m venv .venv
source .venv/bin/activate
python --version    # Should show Python 3.11.x
pip --version
```

Install all dependencies

```
pip install --upgrade pip

# Runtime
pip install \
    fastapi==0.111.0 "uvicorn[standard]==0.29.0" \
    "sqlalchemy[asyncio]==2.0.30" asyncpg==0.29.0 \
    alembic==1.13.0 "redis[hiredis]==5.0.4" \
    pydantic==2.7.0 pydantic-settings==2.3.0 \
    orjson==3.10.3 httpx==0.27.0 \
    python-dateutil==2.9.0 structlog==24.1.0 \
    python-dotenv==1.0.1 stix2==3.0.1 apscheduler==3.10.4

# Dev/test
pip install \
    pytest==8.2.0 pytest-asyncio==0.23.6 pytest-cov==5.0.0 \
    fakeredis==2.23.0 aioredis==2.0.0 ruff==0.4.4

# For Sigma import script
pip install pyyaml
```

Create directory structure

```
mkdir -p threatos/{models,core,ingestion,detection,services,api/routers,workers}
mkdir -p threatos/{migrations/versions,tests/{unit,integration,fixtures},frontend/src/{lib,hooks}}
mkdir -p docs scripts

# Create all __init__.py files
find threatos -type d | xargs -I{} touch {}/__init__.py
```

5. Config Files

.env

```
cat > /opt/threatos/.env << 'EOF'
APP_ENV=production
APP_DEBUG=false
DATABASE_URL=postgresql+asyncpg://threatos:threatos_secret@localhost:5432/threatos
DATABASE_POOL_SIZE=10
DATABASE_MAX_OVERFLOW=20
REDIS_URL=redis://localhost:6379/0
REDIS_STREAM_NAME=threatos:events:normalized
REDIS_CONSUMER_GROUP=detection-workers
REDIS_CONSUMER_NAME=worker-1
WORKER_BATCH_SIZE=500
WORKER_BLOCK_MS=200
ASSET_CRITICALITY_TTL=300
COVERAGE_REFRESH_INTERVAL_SECONDS=3600
ATTCK_BUNDLE_URL=https://raw.githubusercontent.com/mitre/cti/master/enterprise-attack/enterprise-att ...
NMAP_BINARY_PATH=/usr/bin/nmap
NMAP_DEFAULT_TIMEOUT=300
EOF
```

alembic.ini — key line

```
# In alembic.ini set:
sqlalchemy.url = postgresql+asyncpg://threatos:threatos_secret@localhost:5432/threatos

# script_location must point to the migrations folder:
script_location = threatos/migrations
```

pyproject.toml

```
cat > pyproject.toml << 'EOF'
[project]
name = "threatos"
version = "0.1.0"
requires-python = ">=3.11"

[tool.pytest.ini_options]
asyncio_mode = "auto"
testpaths = ["threatos/tests"]
EOF
```

6. Database Migration

Creates all 8 tables in PostgreSQL via Alembic.

Run migration

```
cd /opt/threatos
source .venv/bin/activate

# Install the package so imports work
pip install -e .

# Run migration
alembic upgrade head

# Verify – should show 9 rows (8 tables + alembic_version)
psql -h localhost -U threatos -d threatos -c "\dt"
```

Note: Expected output: alerts, assets, attack_chains, coverage_matrix, detection_rules, purple_team_runs, raw_events, scan_results

7. Source Code Deployment

Deploy all Python source files. The complete codebase is in the `threatos_v2_complete.zip` download. Extract it and copy the `threatos/` directory to `/opt/threatos/`.

From the downloaded zip

```
# If deploying from zip:
cd /tmp
unzip threatos_v2_complete.zip
cp -r threatos_v2/threatos/* /opt/threatos/threatos/
cp threatos_v2/pyproject.toml /opt/threatos/
cp threatos_v2/alembic.ini /opt/threatos/
cp threatos_v2/.env.example /opt/threatos/.env # then edit .env

cd /opt/threatos
source .venv/bin/activate
pip install -e .

# Verify imports work
python -c "from threatos.main import app; print('OK!')"
```

Critical fix — JSONBCompat type decorator

Important: The `JSONBCompat TypeDecorator` must use `load_dialect_impl()` to return a real `JSONB` type on PostgreSQL. Without this, `asyncpg` throws `DatatypeMismatchError` on every `INSERT`.

```
# In threatos/models/raw_event.py, the JSONBCompat class must have:
def load_dialect_impl(self, dialect):
    if dialect.name == "postgresql":
        from sqlalchemy.dialects.postgresql import JSONB
        return dialect.type_descriptor(JSONB())
    return dialect.type_descriptor(Text())

# process_bind_param returns value as-is for postgresql
# (asyncpg handles dict/list natively via JSONB descriptor)
```

Critical fix — UUID primary keys must be str not uuid.UUID

Important: `asyncpg` requires `VARCHAR` columns receive `str` values. All model id columns must use `default=lambda: str(uuid.uuid4())` with `Mapped[str]` type annotation, not `Mapped[uuid.UUID]`.

8. Start the API

Start uvicorn (before systemd setup)

```
cd /opt/threatos
source .venv/bin/activate

# Use port 8001 if 8000 is taken
nohup uvicorn threatos.main:app \
  --host 0.0.0.0 --port 8001 \
  --workers 1 --log-level info \
  > /var/log/threatos-api.log 2>&1 &

sleep 3
curl -s http://localhost:8001/docs | head -3 # Should return HTML
```

9. Verify All Routes

Check all 27 routes are registered

```
curl -s http://localhost:8001/openapi.json | python3 -c "  
import json, sys  
api = json.load(sys.stdin)  
for path in sorted(api['paths'].keys()):  
    methods = list(api['paths'][path].keys())  
    print(f' {str(methods):30s} {path}')
```

Note: Should show 27 routes across /api/ingest, /api/rules, /api/alerts, /api/coverage, /api/assets, /api/chains, /api/purple, /api/scans, /ws

10. Start the Ingest Worker

Start worker (before systemd setup)

```
cd /opt/threatos
source .venv/bin/activate

nohup python -m threatos.workers.ingest_worker \
    > /var/log/threatos-worker.log 2>&1 &

sleep 3
tail -5 /var/log/threatos-worker.log
# Expected: INFO Loaded N rules into engine
# Expected: INFO Worker ready – stream=threatos:events:normalized
```

11. systemd Service Units

Stop the manually started processes and replace with systemd-managed services that auto-start on reboot.

Stop manual processes

```
pkill -f "uvicorn threatos.main" 2>/dev/null
pkill -f "threatos.workers.ingest_worker" 2>/dev/null
sleep 2
```

threatos-api.service

```
cat > /etc/systemd/system/threatos-api.service << 'EOF'
[Unit]
Description=ThreatOS FastAPI
After=network.target postgresql-15.service redis.service
Requires=postgresql-15.service redis.service

[Service]
Type=exec
User=root
WorkingDirectory=/opt/threatos
Environment="PATH=/opt/threatos/.venv/bin:/usr/local/bin:/usr/bin:/bin"
ExecStart=/opt/threatos/.venv/bin/uvicorn threatos.main:app \
    --host 0.0.0.0 --port 8001 --workers 2 --log-level info
Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
EOF
```

threatos-worker.service

```
cat > /etc/systemd/system/threatos-worker.service << 'EOF'
[Unit]
Description=ThreatOS ingest worker
After=network.target threatos-api.service redis.service

[Service]
Type=exec
User=root
WorkingDirectory=/opt/threatos
Environment="PATH=/opt/threatos/.venv/bin:/usr/local/bin:/usr/bin:/bin"
ExecStart=/opt/threatos/.venv/bin/python -m threatos.workers.ingest_worker
Restart=always
RestartSec=5
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
EOF
```

threatos-coverage.service

```
cat > /etc/systemd/system/threatos-coverage.service << 'EOF'
[Unit]
Description=ThreatOS coverage worker
After=network.target threatos-api.service

[Service]
Type=exec
User=root
WorkingDirectory=/opt/threatos
Environment="PATH=/opt/threatos/.venv/bin:/usr/local/bin:/usr/bin:/bin"
ExecStart=/opt/threatos/.venv/bin/python -m threatos.workers.coverage_worker
Restart=always
RestartSec=10
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
EOF
```

Enable and start all services

```
systemctl daemon-reload
systemctl enable threatos-api threatos-worker threatos-coverage
systemctl start threatos-api threatos-worker threatos-coverage
sleep 4

# Verify all three are active
systemctl status threatos-api threatos-worker threatos-coverage \
  --no-pager | grep -E "Active|●"
```

12. Web UI — React Build

Node.js 20 was already installed. Build the React UI and serve via nginx. The built dist/ folder is only ~5MB.

Scaffold and build

```
cd /opt/threatos/threatos

# Scaffold into temp dir (existing src/ blocks overwrite)
npm create vite@latest frontend-new -- --template react-ts
# Press Enter through any prompts

# Move our hooks/api into scaffold
cp -r frontend/src/hooks frontend-new/src/
cp -r frontend/src/lib frontend-new/src/

# Replace old frontend
rm -rf frontend
mv frontend-new frontend
cd frontend

# Install dependencies
npm install axios @tanstack/react-query react-router-dom recharts lucide-react clsx
```

Build for production

```
cd /opt/threatos/threatos/frontend
npm run build

# Output: dist/ folder with ~5MB of static files
ls dist/

# Fix browser tab title
sed -i 's|<title>.*</title>|<title>ThreatOS</title>|' dist/index.html
```

Note: After any UI code change, run `npm run build` again. nginx serves from dist/ directly — no nginx restart needed.

13. nginx Configuration

Create nginx config

```
cat > /etc/nginx/conf.d/threatos.conf << 'EOF'
server {
    listen 8080;
    server_name _;

    root /opt/threatos/threatos/frontend/dist;
    index index.html;

    # React SPA – all routes fall back to index.html
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Proxy API to FastAPI
    location /api/ {
        proxy_pass          http://127.0.0.1:8001;
        proxy_http_version 1.1;
        proxy_set_header    Host $host;
        proxy_set_header    X-Real-IP $remote_addr;
        proxy_read_timeout  300;
    }

    # WebSocket for real-time alerts
    location /ws/ {
        proxy_pass          http://127.0.0.1:8001;
        proxy_http_version 1.1;
        proxy_set_header    Upgrade $http_upgrade;
        proxy_set_header    Connection "upgrade";
        proxy_set_header    Host $host;
        proxy_read_timeout  86400;
    }
}
EOF

nginx -t          # Test config
systemctl reload nginx

# Verify
curl -s http://localhost:8080/ | head -3 # Should return HTML
curl -s http://localhost:8080/api/rules  # Should return JSON
```

Note: Using port 8080 because ports 80/443 were occupied by another app. Change listen 8080 to listen 80 on a fresh server.

14. Sigma Rules Import

Downloads 2,971 rules from SigmaHQ, converts ATT&CK-mapped; rules to ThreatOS detection_ast format, and imports via the API. Result: 2,641 rules imported, 49% ATT&CK; coverage.

Install dependency and run

```
cd /opt/threatos
source .venv/bin/activate
pip install pyyaml

# Dry run first (downloads but does not import)
python scripts/import_sigma_rules.py --dry-run

# Real import (~2 minutes)
python scripts/import_sigma_rules.py

# Expected output:
#   Parsed (ATT&CK-mapped): 2641
#   Imported:                2641
#   Already existed (skip): 0
#   No ATT&CK tag (skip):    330
#   Errors:                  0
#   Coverage: 339/691 techniques (49.1%)
```

Re-run after second import (idempotent)

```
# Second run skips existing rules – safe to run at any time
python scripts/import_sigma_rules.py
# Expected:
#   Imported:                0
#   Already existed (skip): 2641
```

15. Daily Automation (systemd Timer)

Runs the Sigma import every night at 02:00 to pick up new community rules.

Create service and timer

```
cat > /etc/systemd/system/threatos-sigma-import.service << 'EOF'
[Unit]
Description=ThreatOS daily Sigma rule import
After=network.target threatos-api.service

[Service]
Type=oneshot
User=root
WorkingDirectory=/opt/threatos
Environment="PATH=/opt/threatos/.venv/bin:/usr/local/bin:/usr/bin:/bin"
ExecStart=/opt/threatos/.venv/bin/python scripts/import_sigma_rules.py
StandardOutput=journal
StandardError=journal
EOF

cat > /etc/systemd/system/threatos-sigma-import.timer << 'EOF'
[Unit]
Description=Run ThreatOS Sigma import daily at 02:00

[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true
RandomizedDelaySec=300

[Install]
WantedBy=timers.target
EOF

systemctl daemon-reload
systemctl enable --now threatos-sigma-import.timer

# Verify
systemctl list-timers threatos-sigma-import.timer
```

16. Smoke Test — End-to-End Pipeline

Verify the full pipeline: stream event → worker → alert in DB → visible in UI.

Create a detection rule

```
curl -s -X POST http://localhost:8001/api/rules \
-H "Content-Type: application/json" \
-d '{
  "name": "PowerShell EncodedCommand",
  "technique_id": "T1059.001",
  "tactic": "execution",
  "severity": 8,
  "confidence": 0.9,
  "detection_ast": {
    "type": "and",
    "children": [
      {"type": "field_match", "field": "process", "operator": "contains", "value": "powershell"},
      {"type": "field_match", "field": "command_line", "operator": "contains", "value": "-EncodedCommand" ...
    ]
  }
}'
# Expected: {"id": "...", "name": "PowerShell EncodedCommand"}
```

Register an asset

```
curl -s -X POST http://localhost:8001/api/assets \
-H "Content-Type: application/json" \
-d '{"hostname": "WIN-VICTIM", "criticality": 4, "environment": "production", "os_type": "windows"}'
# Expected: {"id": "...", "hostname": "win-victim", "criticality": 4}
```

Push event to Redis stream

```
python3 -c "
import redis, json
r = redis.Redis(host='localhost', port=6379, decode_responses=True)
r.xadd('threatos:events:normalized', {
  'event_id': 'test-001',
  'log_source': 'winlog',
  'host': 'WIN-VICTIM',
  'user': 'jsmith',
  'process': 'powershell.exe',
  'command_line': 'powershell.exe -EncodedCommand SQBFAFgA',
  'raw_fields': json.dumps({'process': 'powershell.exe'})
})
print('Event pushed')
"
sleep 2
```

Verify alert was created

```
curl -s http://localhost:8001/api/alerts
# Expected: [{"technique_id": "T1059.001", "risk_score": 72.0, "status": "open", ...}]

# risk_score = severity(8) * confidence(0.9) * criticality(4) / 40 * 100 = 72.0
```

Verify all DB tables have data

```
psql -h localhost -U threatos -d threatos -c "  
SELECT 'raw_events'          AS t, COUNT(*) FROM raw_events  
UNION ALL SELECT 'detection_rules', COUNT(*) FROM detection_rules  
UNION ALL SELECT 'alerts',    COUNT(*) FROM alerts  
UNION ALL SELECT 'assets',     COUNT(*) FROM assets;"
```

17. Troubleshooting — Bugs Encountered & Fixed

These bugs were found during the actual deployment and are documented here so they can be fixed immediately on a rebuild.

Bug 1 — JSONBCompat: DatatypeMismatchError on INSERT

asyncpg with PostgreSQL JSONB columns throws: `DatatypeMismatchError`: column "log_sources" is of type jsonb but expression is of type character varying Root cause: `TypeDecorator` with `impl=Text` sends VARCHAR to asyncpg. Fix: Override `load_dialect_impl()` to return real JSONB on PostgreSQL.

```
# In threatos/models/raw_event.py – JSONBCompat class:
def load_dialect_impl(self, dialect):
    if dialect.name == "postgresql":
        from sqlalchemy.dialects.postgresql import JSONB
        return dialect.type_descriptor(JSONB())
    return dialect.type_descriptor(Text())

def process_bind_param(self, value, dialect):
    if value is None: return None
    if dialect.name == "postgresql": return value # asyncpg handles it
    return json.dumps(value) # SQLite needs string
```

Bug 2 — UUID: expected str, got UUID

asyncpg throws: `DataError`: invalid input for query argument \$1: UUID(...) (expected str, got UUID) Root cause: Model id column typed as `Mapped[uuid.UUID]` with `default=uuid.uuid4` passes a UUID object instead of string to `VARCHAR(36)`. Fix: Change type annotation to `Mapped[str]` and default to `lambda: str(uuid.uuid4())`

```
# WRONG – causes DataError:
id: Mapped[uuid.UUID] = mapped_column(String(36), primary_key=True, default=uuid.uuid4)

# CORRECT:
id: Mapped[str] = mapped_column(String(36), primary_key=True, default=lambda: str(uuid.uuid4()))
```

Bug 3 — ATT&CK; bundle: stix2.parse() returns dicts not objects

stix2 library on this server returns plain dicts from `parse()`. Code using `obj.type` fails with: `'dict' object has no attribute 'type'` Fix: Skip `stix2.parse()` entirely — use `resp.json()` and access via `dict.get()`

```
# WRONG – stix2 version dependent:
bundle = stix2.parse(resp.text, allow_custom=True)
for obj in bundle.objects:
    if obj.type != "attack-pattern": continue

# CORRECT – works on all versions:
raw = resp.json()
for obj in raw.get("objects", []):
    if obj.get("type") != "attack-pattern": continue
```

Bug 4 — Coverage refresh: API returns 0 techniques

`POST /api/coverage/refresh` returned `{"total_techniques":0}` even after the coverage worker had loaded 691 techniques. Root cause: The ATT&CK; cache is in-memory. The worker process has the cache but the API process does not share it. Fix: The `/refresh` route must call `load_attck_bundle()` itself if cache is empty.

```
# In coverage_router.py:
async def _ensure_attck_loaded() -> None:
    if get_cache_size() == 0:
        await load_attck_bundle(settings.attck_bundle_url)

@router.post("/refresh")
async def trigger_refresh(db: AsyncSession = Depends(get_db)):
    await _ensure_attck_loaded() # Add this line
    summary = await refresh_coverage(db)
    return {...}
```

Bug 5 — Coverage upsert: platforms JSONB parameter

INSERT into coverage_matrix fails with DataError on the platforms column. Root cause: text() queries do not know the JSONB type. Using :platforms::jsonb causes SQLAlchemy to split it as two bind params. Fix: Use CAST(:platforms AS jsonb) syntax and pass json.dumps() string.

```
# WRONG - SQLAlchemy splits :platforms::jsonb:
VALUES (:tid, ..., :platforms::jsonb, ...)

# CORRECT:
VALUES (:tid, ..., CAST(:platforms AS jsonb), ...)
# And pass: "platforms": json.dumps(tech.platforms if tech else [])
```

Bug 6 — pg_hba.conf: trust line for postgres

The postgres superuser has no password saved and the OS user does not exist. pg_ctl reload cannot run as root. Fix: Add trust line to pg_hba.conf, use kill -HUP for reload, connect via Unix socket (not TCP).

```
# 1. Add to TOP of pg_hba.conf:
local all postgres trust

# 2. Reload via SIGHUP (works as root):
kill -HUP $(cat /var/lib/pgsql/15/data/postmaster.pid | head -1)

# 3. Connect via Unix socket (NOT -h localhost):
psql -U postgres -d postgres -c "SELECT current_user;"

# 4. IMMEDIATELY restore pg_hba.conf after creating DB
```

18. Quick Reference Card

URLs

Web UI	http://YOUR_SERVER_IP:8080
API Docs	http://YOUR_SERVER_IP:8001/docs
API Base	http://YOUR_SERVER_IP:8001/api
WebSocket	ws://YOUR_SERVER_IP:8001/ws/alerts

Service management

```
# Status
systemctl status threatos-api threatos-worker threatos-coverage --no-pager

# Restart all
systemctl restart threatos-api threatos-worker threatos-coverage

# Logs (live)
journalctl -u threatos-api -f
journalctl -u threatos-worker -f
journalctl -u threatos-coverage -f

# Check next Sigma import
systemctl list-timers threatos-sigma-import.timer

# Run Sigma import manually
cd /opt/threatos && source .venv/bin/activate
python scripts/import_sigma_rules.py
```

Database

```
# Connect
psql -h localhost -U threatos -d threatos

# Row counts
SELECT 'raw_events' AS t, COUNT(*) FROM raw_events
UNION ALL SELECT 'detection_rules', COUNT(*) FROM detection_rules
UNION ALL SELECT 'alerts', COUNT(*) FROM alerts
UNION ALL SELECT 'assets', COUNT(*) FROM assets
UNION ALL SELECT 'coverage_matrix', COUNT(*) FROM coverage_matrix
UNION ALL SELECT 'attack_chains', COUNT(*) FROM attack_chains;
```

Push a test event through the pipeline

```

source /opt/threatos/.venv/bin/activate
python3 -c "
import redis, json
r = redis.Redis(host='localhost', port=6379, decode_responses=True)
r.xadd('threatos:events:normalized', {
    'event_id': 'test-001',
    'log_source': 'winlog',
    'host': 'WIN-VICTIM',
    'process': 'powershell.exe',
    'command_line': 'powershell -EncodedCommand abc',
    'raw_fields': json.dumps({'process': 'powershell.exe'}),
})
print('Done – check Alerts page in UI')
"
sleep 2
curl -s http://localhost:8001/api/alerts

```

Rebuild from scratch on a new server

```

# Checklist:
# 1. dnf install python3.11 redis nmap nginx
# 2. Install PostgreSQL 15 or use existing
# 3. Create threatos DB and user (see Step 3)
# 4. mkdir /opt/threatos && python3.11 -m venv .venv
# 5. Extract threatos_v2_complete.zip to /opt/threatos
# 6. source .venv/bin/activate && pip install -e ".[dev]"
# 7. Copy .env.example to .env and edit DATABASE_URL / REDIS_URL
# 8. alembic upgrade head
# 9. Create systemd units (Step 11)
# 10. systemctl enable --now threatos-api threatos-worker threatos-coverage
# 11. Build UI: cd frontend && npm install && npm run build
# 12. Configure nginx (Step 13) and systemctl reload nginx
# 13. python scripts/import_sigma_rules.py
# 14. Create systemd timer (Step 15)
# 15. Run smoke test (Step 16)

```

ThreatOS — Oracle Linux 8

Deployment date	April 23, 2026
Server	Oracle Linux 8.10 kernel 5.15.0 UEK
Python	3.11.7
PostgreSQL	15.x (shared with FIM app)
Redis	5.0.3
API port	8001 (8000 occupied by FIM)
UI port	8080 (80/443 occupied)
Detection rules	2,641 imported from SigmaHQ
ATT&CK; coverage	339/691 techniques (49.1%)
Services	3 systemd services + 1 timer
Daily automation	Sigma rules refresh at 02:00 IST